

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_18

18. Iterator Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Iterator pattern.

GoF Definition

It provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

Concept

Iterators are generally used to traverse a container (or a collection of objects) to access its elements without knowing how the data are stored internally. It is very useful when you need to traverse different kinds of collection objects in a standard and uniform way. Figure 18-1 shows a sample and the most common diagram for an Iterator pattern.

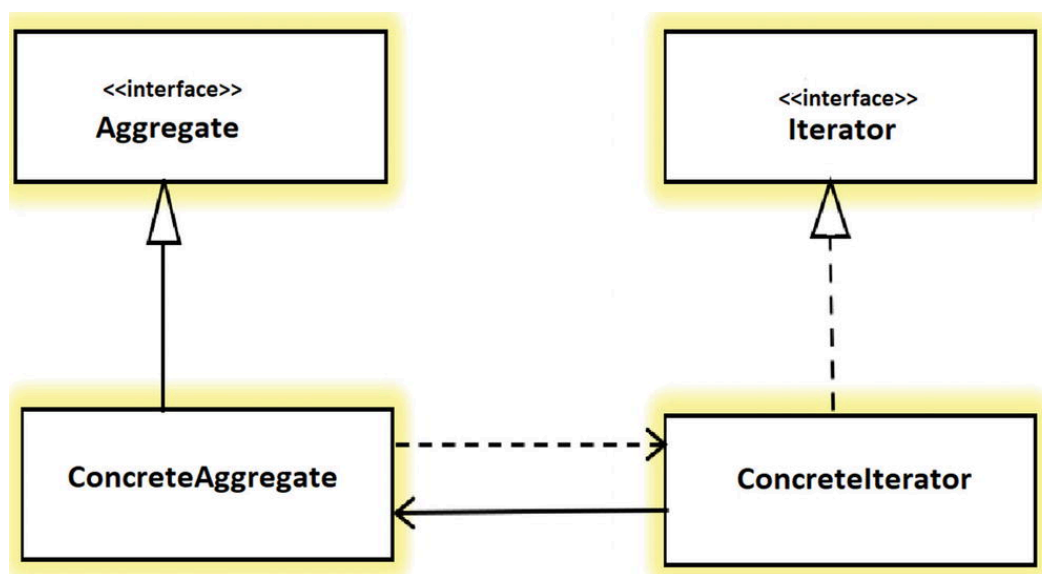


Figure 18-1 A sample diagram for an Iterator pattern

The participants are as follows:

- **Iterator:** It is an interface that is used to access or traverse elements.
- **ConcreteIterator:** It implements the `Iterator` interface methods. It can also keep track of the current position in the traversal of the aggregate.
- **Aggregate:** It defines an interface that can create an `Iterator` object.
- **ConcreteAggregate:** It implements the `Aggregate` interface. It returns an instance of the `ConcreteIterator`.

Points To Note

- This pattern is frequently used to traverse the nodes of a tree-like structure. So, in many examples, you may notice the use of the Iterator pattern with the Composite pattern.
- The role of an iterator is not limited to traversing. This role can vary to support different requirements. For example, you can filter the elements in various ways.
- Clients cannot see the actual traversal mechanism. A client program only uses the iterator methods that are public in nature.

Real-Life Example

Suppose there are two companies, Company A and Company B. Company A stores its employee records (such as each employee's name, address, salary details, and so on) in a linked list data structure, and Company B stores its employee data in an array. One day the two companies decide to merge to form a big organization. The Iterator pattern is handy in such a situation because you do not need to write the code from scratch. In a situation like this, you can have a common interface through which you can access the data for both companies so you can call those methods without rewriting the code.

Consider another example. Suppose your company has decided to promote some employees based on their performances. All the managers get together and set a common criterion for promotion. Then they iterate over the records of the employees one by one to mark the potential candidates for promotion.

Consider an example from a different domain. When you store songs in your preferred audio device, say an MP3 player or your mobile device, you can iterate over them through various button presses or swipe movements. The basic idea is to provide you with some mechanism so that you can go through your list smoothly.

Computer World Example

Go through the following bullet points. They are common examples of the Iterator pattern.

- You may have already used Java's built-in Iterator interface, `java.util.Iterator`. When you use interfaces like `java.util.Iterator` or `java.util.Enumeration`, you basically use this pattern. The concepts of iterators and enumerators have existed for a long time. In simple words, enumerators can produce the next element based on a criterion whereas using iterators, you cycle a sequence from a starting point to the endpoint. In Java, you can treat an `Iterator` as an improved version of an `Enumeration`, because the JDK documentation says the following: `Iterator` takes the place of `Enumeration` in the Java Collections Framework. Iterators differ from enumerations in two ways:
 - `Iterators` allow the caller to remove elements from the underlying collection during the iteration with well-defined semantics.
 - Method names have been improved.
- The `java.util.Scanner` class that implements `Iterator<String>` also follows this pattern. It allows you to iterate over an input stream.

Implementation

In this chapter, you will see three different implementations of the Iterator pattern. I start with an example that follows the core theory of this pattern. In the next example, I modify the example using Java's in-built support of the Iterator pattern. In the third and final example, I show you the usage of this pattern with a different data structure. In the first two examples, I use the String data type, but in the final example, I use a complex data type. Before you start, note the structure of the program in the Package Explorer view for an immediate reference.

Similar to the real-world example, let's assume that in a particular college, a student of the Arts department needs to study four subjects: English, History, Geography, and Psychology. These subjects are stored in an array data structure.

The administrative department does not interfere with how a department maintains the records for its students. It is simply interested in getting the data from each department and wants to access the data uniformly. Now assume you are a member of the administrative department and at the beginning of a new session, you want to write a program to advertise the curriculum using the iterators. Let's see how to implement it in the upcoming demonstration.

The Iterator pattern enables a client object to traverse through a collection of objects (often termed as a container) without having the container reveal how the data is stored internally. This is why, to implement this pattern, you should remember the following points:

- This pattern suggests you have a container object that provides a public interface in the form of an `Iterator` object so that different client objects can access its contents.
- An `Iterator` object contains public methods to allow a client object to navigate through the list of objects within the container.

Let's assume that your iterator currently supports four basic methods named `first()`, `next()`, `currentItem()`, and `hasNext()`. As per their naming,

- The `first()` method reset the pointer to the first element before you start traversing a data structure.
- The `next()` method returns the next element in the container.
- The `currentItem()` method returns the current element of the container where the iterator is pointing at a particular point of time.
- The `hasNext()` method validates whether any next element is available for further processing or not. So, the `hasNext()` method helps you to decide whether you have reached the end of your container or not.

This is why in the upcoming demonstration you see an `Iterator` that acts as a common interface. It is defined as follows:

```
interface Iterator {  
    void first();  
    String next();  
    String currentItem();  
    boolean hasNext();  
}
```

Any class that implements this interface must provide the concrete implementations for these methods. `ArtsIterator` does the same. You may notice that these implementations purely depend on the underlying data structure. For example, you need to define the `currentItem()` method differently if you use a `LinkedList` data structure instead of an array data structure (you'll get the idea when you see the third implementation in this chapter).

To print the curriculum, you need two of these methods, `hasNext()` and `next()`. But you also see the use of the `first()` and `currentItem()` methods. I mention all four methods and provide sample implementations for them because they are very common in different Iterator pattern implementations.

There is another interface called `Department` that has one method, as follows:

```
interface Department {  
    Iterator createIterator();  
}
```

The Arts department uses an array data structure. This is why you see the following class in demonstration 1:

```
class ArtsDepartment implements Department{  
    private String[] subjects;  
    public ArtsDepartment() {  
        subjects = new String[] { "1.English",  
                                   "2.History",  
                                   "3.Geography",  
                                   "4.Psychology"  
                                   };  
    }  
    public Iterator createIterator() {  
        return new ArtsIterator(subjects);  
    }  
}
```

Class Diagram

Figure **18-2** shows the class diagram. In the upcoming example, you will see that an `ArtsDepartment` object creates an `ArtsIterator` object. This is why I edited the

dashed line with an arrowhead (which is used to denote a “usage” link in Papyrus) and attach the tag <<create>> to it for a better understanding.

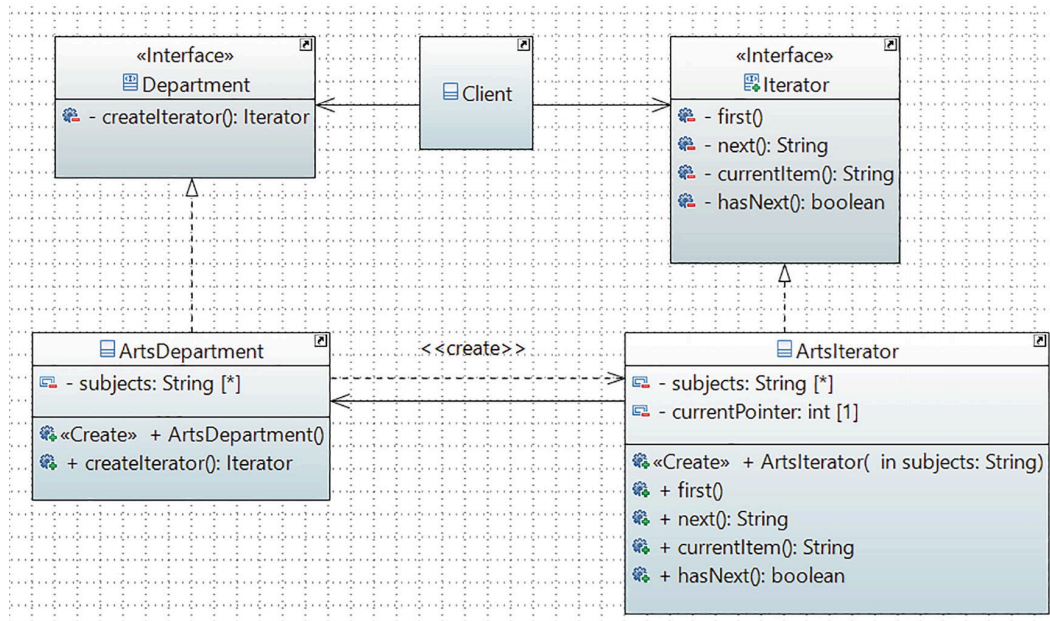


Figure 18-2 Class diagram

Package Explorer View

Figure 18-3 shows the high-level structure of the program.

📁 iterator

- ▼ 📁 implementation_1
 - ▼ 📄 ArtsDepartment.java
 - ▼ 🏠 ArtsDepartment
 - subjects
 - 🏠 ArtsDepartment()
 - 🏠 createIterator() : Iterator
 - ▼ 📄 ArtsIterator.java
 - ▼ 🏠 ArtsIterator
 - currentPointer
 - subjects
 - 🏠 ArtsIterator(String[])
 - 🏠 currentItem() : String
 - 🏠 first() : void
 - 🏠 hasNext() : boolean
 - 🏠 next() : String
 - ▼ 📄 Client.java
 - > 🏠 Client
 - ▼ 📄 Department.java
 - ▼ 🏠 Department
 - 🏠 createIterator() : Iterator
 - ▼ 📄 Iterator.java

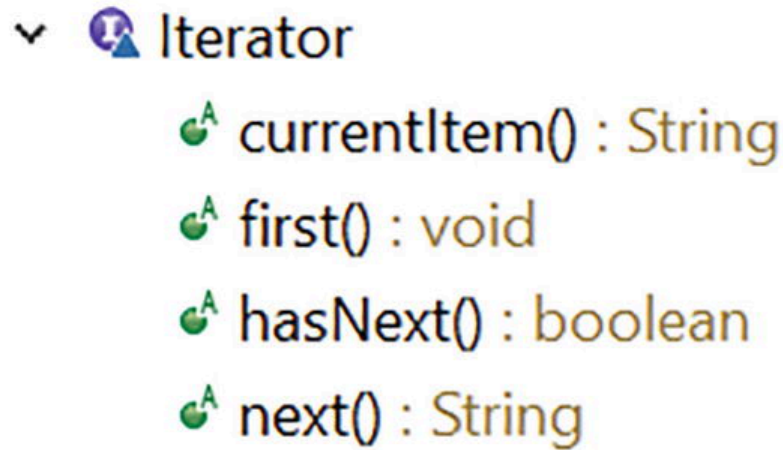


Figure 18-3 Package Explorer view

Demonstration 1

All parts of this demonstration are stored inside the package `jdpe3e.iterator.implementation_1`. Here's the implementation:

```
// Department.java
interface Department {
    Iterator createIterator();
}

// ArtsDepartment.java
class ArtsDepartment implements Department{
    private String[] subjects;
    public ArtsDepartment() {
        subjects = new String[] {
            "1. English",
            "2. History",
            "3. Geography",
            "4. Psychology"
        };
    }
    public Iterator createIterator() {
        return new ArtsIterator(subjects);
    }
}

// Iterator.java
interface Iterator {
    void first(); // Reset to first element
    String next(); // To get the next element
    String currentItem(); // To retrieve the current element
    boolean hasNext(); // To check the next element.
}

// ArtsIterator.java
```



```

class ArtsIterator implements Iterator {
    private String[] subjects;
    private int currentPointer;
    public ArtsIterator(String[] subjects) {
        this.subjects = subjects;
        currentPointer = 0;
    }
    @Override
    public void first() {
        currentPointer = 0;
    }
    @Override
    public String next() {
        // System.out.println("Currently pointing to: "+
        // this.currentItem());
        return subjects[currentPointer++];
    }
    @Override
    public String currentItem() {
        return subjects[currentPointer];
    }
    @Override
    public boolean hasNext() {
        if (currentPointer >= subjects.length)
            return false;
        return true;
    }
    // return currentPointer >= subjects.length? false: true;
}

// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Iterator Pattern Demonstration-1***\n");
        Department arts = new ArtsDepartment();
        Iterator artsIterator = arts.createIterator();
        System.out.println("Iterating over the Arts subjects:\n");
        while (artsIterator.hasNext()) {
            System.out.println(artsIterator.next());
        }
        // Moving back to first element
        artsIterator.first();
        System.out.println("\nThe pointer moves to -> " +
            artsIterator.currentItem());
    }
}

```

Output

Here's the output:

```
***Iterator Pattern Demonstration-1***  
Iterating over the Arts subjects:  
1. English  
2. History  
3. Geography  
4. Psychology  
The pointer moves to -> 1. English
```

Additional Comments

I want you to note the following comments:

- You can see that the `Iterator` interface methods `hasNext()`, `next()`, `first()`, and `currentItem()` are common methods for a client. Any class that implements this interface must provide implementations following its internal data structure, but from the client's point of view, it does not matter because they can uniformly invoke these methods. Since `ArtsDepartment` uses an array, these implementations are based on the array data structure. If another class uses a different data structure, say a `List` or a `LinkedList`, you need to update the code accordingly. Demonstration 3 shows such an example.
- Some developers prefer to use a conditional operator instead of an `if-else` chain. This choice is up to you. I commented out this alternative approach inside the `hasNext()` method for reference.
- Additional use of the `currentItem()` method is shown with commented code inside the `next()` method. If you want to test it, you can uncomment the line.

Demonstration 2

Now you'll see another implementation using Java's built-in `Iterator` interface that supports the `Iterator` pattern.

- Here you use Java's built-in support for the Iterator pattern. Note the inclusion of the following line at the beginning of the program:

```
import java.util.Iterator;
```

- If you open the source code, you will see that `Iterator<E>` interface has three methods named `hasNext()`, `next()`, and `remove()`, as follows:

```
boolean hasNext();  
E next();  
default void remove() {  
    throw new UnsupportedOperationException("remove");  
}
```

- You can see that `remove()` has a default implementation. So, in the following example, you need to override the `hasNext()` and `next()` methods only.
- Here you are using Java's `Iterator` interface so there is no need to define your iterator interface.
- In this program, the `ArtsIterator` class implements `Iterator<String>`. You do not need to write `ArtsIterator<String>`. It is because the type parameter is already mentioned in the generic interface. Java allows you to write this type of code.
- To maintain the similarity with demonstration 1, you use two more methods, `first()` and `currentItem()`, inside the `ArtsIterator` class. So, when you call these methods inside the client code, you need to typecast properly.
- In this modified implementation, key changes are shown in bold.

Since the overall concept and intent are similar to demonstration 1, you can jump directly to demonstration 2. All parts of the programs are stored inside the `jdp3e.iterator.implementation_2` package. Here's the complete implementation:

Do not forget to import `java.util.Iterator` at the beginning of these files. To avoid repetition, I excluded them.

```
// Department.java
```

```
interface Department {
    Iterator<String> createIterator();
}

// ArtsDepartment.java
class ArtsDepartment implements Department {
    private String[] subjects;
    public ArtsDepartment() {
        subjects = new String[] { "1. English",
                                   "2. History",
                                   "3. Geography",
                                   "4. Psychology" };
    }
    @Override
    public Iterator<String> createIterator() {
        return new ArtsIterator(subjects);
    }
}

// ArtsIterator.java
class ArtsIterator implements Iterator<String> {
    private String[] subjects;
    private int currentPointer;
    public ArtsIterator(String[] papers) {
        this.subjects = papers;
        currentPointer = 0;
    }
    public void first() {
        currentPointer = 0;
    }
    public String currentItem() {
        return subjects [currentPointer];
    }
    @Override
    public boolean hasNext() {
        if (currentPointer >= subjects.length)
            return false;
        return true;
    }
    @Override
    public String next() {
        return subjects [currentPointer ++];
    }
}

// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Iterator Pattern Demonstration-2.***)"
```

```
Department arts = new ArtsDepartment();
Iterator<String> artsIterator =
    arts.createIterator();
System.out.println("Iterating over the Arts subjects:\n");
while (artsIterator.hasNext()) {
    System.out.println(artsIterator.next());
}
// Moving back to first element
((ArtsIterator) artsIterator).first();
String pointer= ((ArtsIterator)artsIterator).
    currentItem();
System.out.println("\nThe pointer moves to -> " + pointer);
}
}
```

Output

Here's the output:

```
***Iterator Pattern Demonstration-2.***
Iterating over the Arts subjects:
1. English
2. History
3. Geography
4. Psychology
The pointer moves to -> 1. English
```

Q&A Session

18.1 What is the Iterator pattern used for?

The following includes some usage:

- You can traverse an object structure without knowing its internal details. As a result, if you have a collection of different subcollections (for example, your container is mixed with arrays, lists, linked lists, and so on), you can still traverse the overall collection and deal with the elements in a universal way without knowing the internal details or differences among them.
- You can traverse a collection in different ways. If they are designed properly, multiple traversals are also possible in parallel.

18.2 What are the key challenges associated with this pattern?

You must make sure that no accidental modification has taken place during the traversal procedure.

18.3 But to deal with the challenge mentioned earlier, you can simply make a backup and then proceed. Am I right?

Making a backup and re-examining it later is a costly operation.

18.4 Throughout the discussion, you talked about collections. What is a collection?

It is a group of individual objects that are presented in a single unit. You may often see the use of interfaces like `java.util.Collection` and `java.util.Map` in Java programs. They are common interfaces for Java's collection classes and they are introduced from JDK 1.2 onwards.

Before collections, you had choices like arrays and vectors to store or manipulate a group of objects. But these classes do not have a common interface. The way you access elements in an array is quite different from the way you access the elements of a vector. This is why it's difficult to write a common algorithm to access different elements from these different implementations. Also, many of these methods are final, so you can't extend them.

The collection framework was introduced to address this kind of difficulty. At the same time, it provides high-performance implementations to make programmers' lives easier.

18.5 In the modified implementation, why am I not seeing the `@Override` annotation for the methods `first()` and `currentItem()`?

These two methods are not present in the `java.util.Iterator` interface. The built-in `Iterator` interface has the methods `hasNext()` and `next()`. So, I use the `@Override` annotation for these methods only. Lastly, there is another method called `remove()` in this interface. This method has a default implementation. Since I have not used it, I did not need to modify this method.

18.6 In these implementations, I am seeing that you are using an array of Strings to store and manipulate data. Can you show an Iterator pattern implementation that uses a relatively complex data type and a different data structure?

To make these examples simple, I use strings and an array data structure only. You can always choose your preferred data structure and apply the same process when you consider a complex data type. For example, consider the following illustration (demonstration 3) with these key characteristics:

- Here I use a relatively complex data type named `Employee`. Each employee object has three fields: a name, an identification number(id), and salary. To reduce the code size, I do not use any setter methods in this class, but I use getter methods.
- This time, instead of using an array, I use a different data structure named `LinkedList` in the following implementation. So, I need to include the following line in this implementation:

```
import java.util.LinkedList;
```

- I follow the same approach as the previous example.

Demonstration 3

Here is another demonstration of the Iterator pattern. All parts of the programs are stored inside the `jdp3e.iterator.implementation_3` package. Here's the complete implementation:

```
// Employee.java
/**
 * This is the Employee class.
 * This class has getter methods,
 * but no setter methods.
 */
class Employee {
    private String name;
    private int id;
    private double salary;
    public Employee(String name, int id, double salary) {
        this.name = name;
        this.id = id;
        this.salary = salary;
    }
}
```



```

    }
    public String getName() {
        return name;
    }
    public int getId() {
        return id;
    }
    public double getSalary() {
        return salary;
    }
    @Override
    public String toString() {
        return "Employee Name: " + this.getName() + ", ID: " + this.getId() + " and salary: $" + this.getSalary();
    }
}

// Database.java
import java.util.Iterator;
interface Database {
    Iterator<Employee> createIterator();
}

// EmployeeDatabase.java
import java.util.LinkedList;
class EmployeeDatabase implements Database {
    private LinkedList<Employee> employeeList;
    public EmployeeDatabase() {
        employeeList = new LinkedList<Employee>();
        employeeList.add(new Employee("Ron", 1, 1000.25));
        employeeList.add(new Employee("Jack", 2, 2000.5));
        employeeList.add(new Employee("Sufi", 3, 3000.75));
        employeeList.add(new Employee("Jian", 4, 2550.0));
        employeeList.add(new Employee("Alex", 5, 753.83));
    }
    public EmployeeIterator createIterator() {
        return new EmployeeIterator(employeeList);
    }
}

// EmployeeIterator.java
import java.util.Iterator;
import java.util.LinkedList;
class EmployeeIterator implements Iterator<Employee> {
    private LinkedList<Employee> employeeList;
    private int currentPointer;
    public EmployeeIterator(LinkedList<Employee> employeeList) {

```

```

        this.employeeList = employeeList;
        currentPointer = 0;
    }
    public void first() {
        currentPointer = 0;
    }
    public Employee currentItem() {
        return employeeList.get(currentPointer);
    }
    @Override
    public Employee next() {
        return employeeList.get(currentPointer ++);
    }
    @Override
    public boolean hasNext() {
        if (currentPointer >= employeeList.size())
            return false;
        return true;
    }
}

// Client.java
import java.util.Iterator;
class Client {
    public static void main(String[] args) {
        System.out.println("***Iterator Pattern Demonstration-3***")
        Database employeesList = new EmployeeDatabase();
        Iterator<Employee> employeeIterator =
            employeesList.createIterator();
        System.out.println("Iterating over the employee list:\n");
        while (employeeIterator.hasNext()) {
            System.out.println(employeeIterator.next());
        }
        // Moving back to first element
        // artsIterator.first();
        ((EmployeeIterator) employeeIterator).first();
        Employee pointer = ((EmployeeIterator)
            employeeIterator).currentItem();
        System.out.println("\nThe pointer moves to -> " + pointer);
    }
}

```

Output

Here's the output:

```
***Iterator Pattern Demonstration-3***
```

```
Iterating over the employee list:
```

```
Employee Name: Ron, ID: 1 and salary: $1000.25
```

```
Employee Name: Jack, ID: 2 and salary: $2000.5
```

```
Employee Name: Sufi, ID: 3 and salary: $3000.75
```

```
Employee Name: Jian, ID: 4 and salary: $2550.0
```

```
Employee Name: Alex, ID: 5 and salary: $753.83
```

```
The pointer moves to -> Employee Name: Ron, ID: 1 and salary: $1000.25
```

Previous chapter

< [17. Chain of Responsibility Pattern](#)

Next chapter

[19. Command Pattern](#) >